

Important: Not Available in notes4.md

This file contains only the parts from `preparation.md` that were marked as not available, not directly available, or only inferred from `notes4.md`.

Question 14: MPI communication and mesh network

Original question/scenario:

A climate simulation application is running on a distributed parallel computing system consisting of 16 processors, where each processor has its own local memory. The processors are connected using a mesh interconnection network.

The application divides a large dataset into equal parts, and each processor performs computations on its portion. After each computation step, processors must exchange boundary data with their neighboring processors. At the end of execution, all partial results are sent to a master processor, which computes the final result.

However, the system is experiencing:

- High communication delay
- Reduced efficiency as the number of processors increases

Tasks:

1. Identify the parallel computing model used in this system.
2. Explain how MPI is used for communication in this scenario.
3. Identify the type of interconnection network and explain its working.
4. Discuss the information exchange technique used.
5. Analyze the performance issues such as latency and efficiency.
6. Suggest improvements to enhance performance.

Not available in `notes4.md`: MPI is not directly explained.

MPI stands for Message Passing Interface. In the climate simulation scenario, processors use MPI to exchange data because each processor has its own local memory. Neighboring processors exchange boundary data after each computation step. At the end, all processors send partial results to a master processor.

Partially not available in `notes4.md`: mesh networks are not explained in detail.

A mesh interconnection network connects each processor to nearby neighboring processors. This works well for climate simulation because each processor mainly needs boundary data from adjacent regions.

Question 16: FinTech-specific Flynn's Taxonomy examples

Original question/scenario:

A financial technology (FinTech) company is developing high-performance computing systems to process large volumes of transactional and market data. Their applications include real-time fraud detection during online payments, batch processing of historical trading data, and predictive analytics for stock market trends and risk assessment.

The company needs to select appropriate computer architectures that can efficiently support these varied workloads.

Tasks:

1. Explain Flynn's Classification of computer architectures.
2. Describe each category with examples relevant to financial technology applications.
3. Analyze which architecture is most commonly used in modern FinTech computing systems and justify your answer.

Not available in `notes4.md`: FinTech-specific examples are not given.

- SISD: A simple program processing one transaction at a time.
- SIMD: Market-data vector calculations, risk scoring over many records, or GPU-based analytics.
- MISD: Theoretical redundant checking of the same transaction data through different validation methods.
- MIMD: Fraud detection, batch processing of trading history, payment processing, and predictive analytics.

Most modern FinTech systems commonly use MIMD because they depend on multicore CPUs, clusters, and distributed servers. SIMD accelerators may also be used for matrix-heavy analytics.

Question 17: Robotics-specific architecture recommendation

Original question/scenario:

A robotics manufacturing company is designing an advanced autonomous production system. The system includes multiple components:

- A central control unit that handles general decision-making and system coordination.
- A vision processing module that analyzes images from multiple cameras simultaneously for quality inspection.
- A high-speed numerical computation unit used for matrix operations in robotic motion and simulation.
- Different embedded controllers are being considered for real-time tasks, with varying instruction complexity and performance needs.

The engineering team is evaluating different types of microprocessors and instruction set architectures to choose the most suitable processors for each module.

Tasks:

1. Explain what a microprocessor is.
2. Describe the types of microprocessors: Scalar, Vector, Array, and explain their use cases.
3. Differentiate between CISC and RISC architectures.
4. Recommend which type(s) of microprocessor architecture should be used in each part of the system and justify your choice.

Not directly available in `notes4.md`: the exact robotics scenario is not given, so this recommendation is inferred from the notes.

- Central control unit: use a scalar or superscalar general-purpose processor because it handles decision-making, control flow, and coordination.
- Vision processing module: use a SIMD array processor or GPU-style architecture because images contain many pixels that can be processed with the same operation.
- High-speed numerical computation unit: use vector or array processors because matrix operations are data-parallel.
- Embedded real-time controllers: use RISC because it gives fast response, simpler instruction decoding, pipelining support, and low power consumption.
- If compatibility with older software or compact code is required, CISC can be considered, but it is less suitable for low-power real-time embedded control.

Question 18: Healthcare-specific Flynn's Taxonomy examples

Original question/scenario:

A healthcare analytics company is building computing systems for processing large-scale patient data. The applications include real-time monitoring of patients' vital signs, large-scale medical image analysis, and predictive analytics for disease outbreaks.

The company wants to choose suitable architectures to handle these diverse workloads efficiently.

Tasks:

1. Explain Flynn's Classification of computer architectures.
2. Describe each category with examples relevant to healthcare applications.
3. Analyze which architecture is most commonly used in modern healthcare computing systems and justify your answer.

Not available in `notes4.md`: healthcare-specific examples are not given.

- SISD: A standalone program processing one patient's record sequentially.
- SIMD: Medical image analysis, where the same image-processing operation is applied to many pixels.
- MISD: Theoretical fault-tolerant checking of the same patient signal by different algorithms.
- MIMD: Real-time monitoring, disease-outbreak prediction, hospital databases, and distributed analytics systems.

Most modern healthcare analytics systems commonly use MIMD because they often depend on multicore CPUs, clusters, and distributed systems. SIMD can also be used for specialized image-processing workloads.

Question 19: Attached array processor limitation

Original question/scenario:

A weather forecasting research center is designing a high-performance computing system to process real-time satellite images, climate simulation data, and atmospheric readings. The system must handle large datasets in parallel and perform repetitive mathematical operations efficiently.

The engineers are considering two architectures:

- SIMD Array Processor
- Attached Array Processor

They are unsure which architecture is more suitable for different parts of the system.

Tasks:

1. Explain SIMD Array Processor.
2. Explain Attached Array Processor.
3. Draw simple diagrams for both architectures.
4. Compare both architectures and state their limitations.
5. Recommend which is better for weather forecasting systems and justify your answer.

Partially not available in `notes4.md`: limitations of attached array processors are only lightly covered.

An attached array processor can be limited by host communication and I/O transfer overhead. Since it works as an auxiliary processor connected to a host computer, performance may depend on how quickly data can move between the host, the I/O interface, the attached processor, and local memory.

Question 22: Client-server architecture in a hospital system

Original question/scenario:

A hospital introduces a system where patient data is accessed simultaneously by doctors, labs, and billing departments. Illustrate how client-server architecture supports this parallel usage.

Not available in `notes4.md`: client-server architecture is not explained.

In a hospital client-server system, the central server stores and manages patient data. Doctors, laboratories, and billing departments act as clients that send requests to the server.

This supports parallel usage because many clients can access the system at the same time. For example:

- A doctor can view patient history.
- A lab can upload test results.
- Billing can update payment records.
- The server coordinates access to shared patient data.

To keep the data correct, the server must use synchronization, access control, and transaction management so that simultaneous updates do not corrupt patient records.

Question 23: DNA sequence comparison example

Original question/scenario:

A research lab wants to speed up DNA sequence comparison by using multiple processors at once. Discuss how parallel computing improves the performance of this scientific task.

Not directly available in `notes4.md`: DNA-specific examples are not given, so the example is inferred from the notes.

Parallel computing improves DNA sequence comparison by dividing the large comparison task into smaller parts and processing them at the same time on multiple processors.

For example, different processors can compare different sequence segments. SIMD or GPU-style processing can apply the same comparison operation across many data items. MIMD systems can also run different parts of the analysis on different datasets.

The final results can be combined after all processors finish their assigned work.

Question 24: Distributed system models for ride-sharing

Original question/scenario:

A ride-sharing company like Uber/Careem uses real-time location updates from thousands of drivers. Describe how distributed system models, including peer-to-peer, master-slave, and hybrid, can be used to manage such real-time operations.

Not available in `notes4.md`: peer-to-peer, master-slave, and hybrid distributed system models are not explained.

Peer-to-peer model:

- Nodes communicate directly with each other.
- Nearby drivers and users may exchange local updates through nearby services.
- It reduces dependence on one central point, but coordination and security are harder.

Master-slave model:

- A central master server coordinates the system.
- Worker servers process driver locations, ride requests, pricing, and route data.
- It is easier to control, but the master can become a bottleneck if not replicated.

Hybrid model:

- Combines central coordination with distributed regional servers.
- Regional servers handle local driver updates and ride matching.
- Central servers manage global data, payments, policies, and monitoring.

For Uber/Careem-like systems, a hybrid distributed model is usually best because it supports real-time regional processing while still keeping central control.